

SPRAWOZDANIE

Na dzisiejszych laboratoriach zajmowaliśmy się generowaniem zbioru Mandelbrota z wykorzystaniem wielowątkowości. Wykonałem pomiary czasu dla rosnącej liczby wątków 1,2,4,8 256,512. Dodatkowo wygenerowałem obraz zbioru Mandelbrota, w którym każdy wątek został oznaczony innym kolorem, aby wizualnie pokazać, które fragmenty obrazu były przetwarzane przez poszczególne wątki.

Kod wyliczania czy punkt należy do zbioru mandelbrota

```
void mandelbrot(int tid){
    int start=(iYmax/num_threads)*tid;
    int stop=start+(iYmax/num_threads);

    for(int iY=start;iY<stop;iY++)
    {
        double Cy=CyMin + iY*PixelHeight;
        if (fabs(Cy)< PixelHeight/2) Cy=0.0;
        int Iteration=0;

        for(int iX=0;iX<iXmax;iX++)
        {
            double Cx=CxMin + iX*PixelWidth;
            double Zx=0.0;
            double Zy=0.0;
            double Zx2=Zx*Zx;
            double Zy2=Zy*Zy;
            for (Iteration=0;Iteration<IterationMax && ((Zx2+Zy2)<ER2);Iteration++)
            {
                Zy=2*Zx*Zy + Cy;
                Zx=Zx2-Zy2 +Cx;
                Zx2=Zx*Zx;
                Zy2=Zy*Zy;
            };
            sum[tid]+=Iteration;
            if (Iteration==IterationMax)
            {
                color[iY][iX][0]=255;
                color[iY][iX][1]=255;
                color[iY][iX][2]=255;
            }
            else
            {
                int base = (tid * 40) % 256;
                color[iY][iX][0]=0;
                color[iY][iX][1]=base/2;
                color[iY][iX][2]=100+base;
            }
        }
    }
}
```

Wyniki czasowe i obserwacje

	A	B	C	D	E	F	G	H	I	J	K
N		1	2	4	8	16	32	64	128	256	512
METODA PODZIAŁ NA BLOKI	16.9677	8.4067	8.54225	7.30516	4.87363	3.47504	3.00439	2.87545	2.85453	2.85595	

Jak widać na wykonanych pomiarach zauważyłem, że czas obliczeń maleje wraz ze wzrostem liczby wątków, nawet powyżej liczby fizycznych wątków procesora. Na ostatnich laboratoriach zauważyliśmy, że przy moim procesorze Intel Core i5-8250U CPU posiadający 4 procesy i 8 wątków. Po 8 wątkach czas przestał się poprawiać. Natomiast tutaj pomimo większej ilości wątków niż fizycznych wątków procesora czas ulega znacznej poprawie.

Wyjaśnienie zjawiska

W celu dojścia do wyjaśnienia tego fenomenu obliczyłem liczbę wykonanych iteracji przez każdy wątek oraz stworzyłem kolorową wizualizację pokazującą, które obszary obrazu były przetwarzane przez poszczególne wątki.

Suma iteracji dla poszczególnych wątków

```
Dla 4  
Dla wątek 1 Suma iteracji:46717753  
Dla wątek 2 Suma iteracji:1108029526  
Dla wątek 3 Suma iteracji:1109165587  
Dla wątek 4 Suma iteracji:46738483  
8.54225
```

Obraz zbioru Mandelbrota

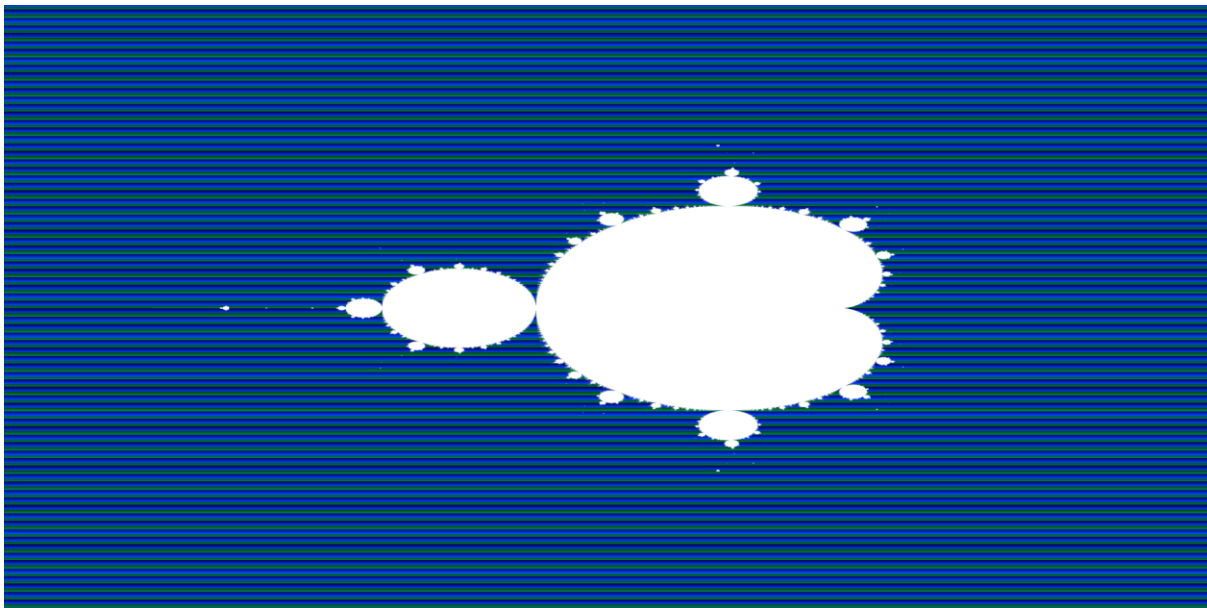


Na podstawie uzyskanych danych widać, że wątki 1 i 4 wykonały znacznie mniej iteracji. Oznacza to, że miały mniej pracy, ponieważ przetwarzały obszary, które nie należały do zbioru Mandelbrota. Z kolei wątki 2 i 3 otrzymały trudniejsze fragmenty obrazu, co spowodowało nierównomierne rozłożenie obciążenia. Na załączonym obrazku widać, że wątek 1 (czarny) i 4 (zielony) nie trafiały na zbiór mandelbrota przez co wykonywały mniej pracy. Przy większej ilości wątków czas przyspieszył ponieważ wątki rozdzielili swoją pracę o wiele bardziej równomiernie co spowodowało zwiększenie efektywności programu. To pokazuje, że w niektórych przypadkach statyczny podział pracy (np. po wierszach) różne wątki mogą być nierówno obciążone.

Suma iteracji pierwszych wątków przy wykorzystaniu 512 wątków

```
Dla 512
Dla wątko 1 Suma iteracji:216369
Dla wątko 2 Suma iteracji:226650
Dla wątko 3 Suma iteracji:233158
Dla wątko 4 Suma iteracji:238398
Dla wątko 5 Suma iteracji:242904
Dla wątko 6 Suma iteracji:246912
Dla wątko 7 Suma iteracji:250556
Dla wątko 8 Suma iteracji:253926
Dla wątko 9 Suma iteracji:257048
Dla wątko 10 Suma iteracji:259996
Dla wątko 11 Suma iteracji:262780
Dla wątko 12 Suma iteracji:265424
Dla wątko 13 Suma iteracji:267946
Dla wątko 14 Suma iteracji:270360
Dla wątko 15 Suma iteracji:272686
```

Obraz dla 512 wątków wątki równomiernie rozłożone



Jak rozwiązać problem?

Mój algorytm dzieli wiersze statycznie pomiędzy wątki czyli np. przy 100 wierszach pierwszy wątek będzie miał wiersze od 1 do 25. Co powoduję nierównomierne rozłożenie pracy. Aby poprawić wydajność, zastosowałem podział naprzemienny (tzw. podział roletowy), w którym wątki otrzymują wiersze na przemian. W tym celu zmodyfikowałem główną pętlę programu tak, aby każdy wątek pobierał co n -ty wiersz obrazu.

Modyfikacja głównej pętli for (wiersze na przemian)

```
for(int iY=tid;iY<iYmax;iY+=num_threads)
{
    double Cy=CyMin + iY*PixelHeight;
    if (fabs(Cy)< PixelHeight/2) Cy=0.0;
    int Iteration=0;
    Czasy
    for(int iX=0;iX<iXmax;iX++)
    {
        double Cx=CxMin + iX*PixelWidth;
        double Zx=0.0;
    }
}
```

Czasy porównanie

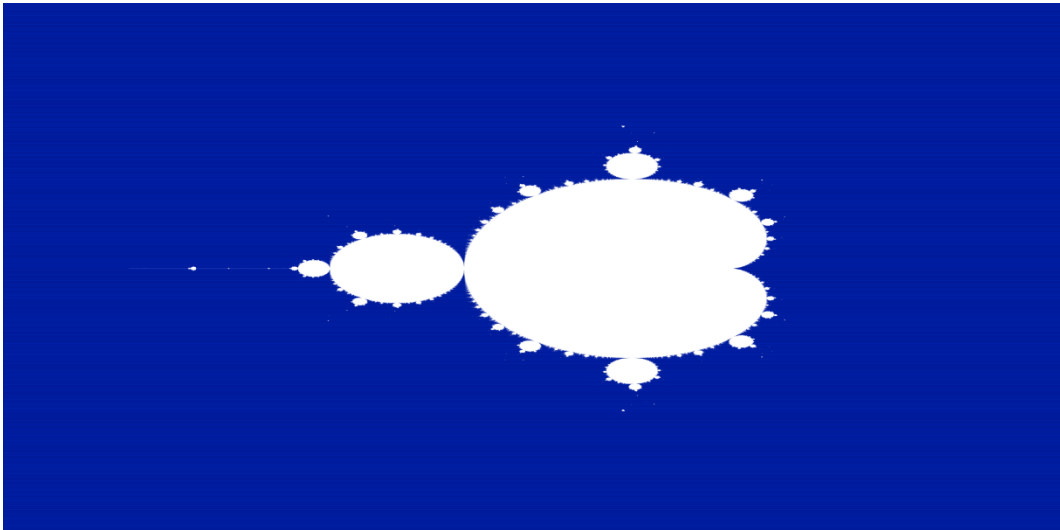
N	1	2	4	8	16	32	64	128	256	512
METODA PODZIAŁ NA BLOKI	16.9677	8,4067	8.54225	7.30516	4.87363	3.47504	3.00439	2.87545	2.85453	2.85595
METODA ROLETY	17.4244	9,68922	5.79364	3.66359	3,3541	3.15754	3.18418	3.29687	3.06062	2,8987

Suma iteracji

```
Dla 4
Dla wątko 1 Suma iteracji:577831452
Dla wątko 2 Suma iteracji:577603601
Dla wątko 3 Suma iteracji:577612699
Dla wątko 4 Suma iteracji:577603597
5.79364

Dla 8
Dla wątko 1 Suma iteracji:289055947
Dla wątko 2 Suma iteracji:288792828
Dla wątko 3 Suma iteracji:288806346
Dla wątko 4 Suma iteracji:288810770
Dla wątko 5 Suma iteracji:288775505
Dla wątko 6 Suma iteracji:288810773
Dla wątko 7 Suma iteracji:288806353
Dla wątko 8 Suma iteracji:288792827
3.66359
```

Obraz



Przybliżenie na rozłożenie kolorów (wątków)



Omówienie wyników

Dla 4 wątków i zwłaszcza dla 8 wątków czasy zmalały i nie różnią się wyniki zbytnio od większej ilości wątków. Tym razem liczba wykonanych iteracji dla każdego watka jest rozłożona równomiernie bez odchyłeń co powoduje wysoką wydajność, pracę każdego watka. Na zwizualizowanym obrazie widać, że wątki pobierają obszary naprzemiennie. Co powoduje wygląd tła podobny do gradientu.

Rozwiązanie problemu za pomocą mutex

Innym sposobem rozwiązania problemu jest podział dynamiczny czyli ten wątek co nie wykonuje zadania bierze następne zadanie (w tym przykładzie wiersz). Każdy wątek otrzymywał z góry ustaloną część wierszy do przetworzenia co może prowadzić do nierównomiernego obciążenia. W celu zaimplementowania podziału dynamicznego zastosowałem zmienną współdzieloną counter która będzie przechowywała numer kolejnego wiersza do przetworzenia. Counter będzie zmieniany w sekcji krytycznej w mechanizmie synchronizacji w tym przypadku mutexa. Mutex to obiekt, który pozwala na jednoczesny dostęp do danej sekcji kodu tylko jednemu wątkowi. Jak dany wątek wejdzie do tej sekcji (tzw. Krytycznej) pozostałe wątki muszą czekać, aż ten wątek zakończy swoje działanie w tej sekcji. Sekcja krytyczna to fragment kodu, w którym wykonywane operacje na danych są współdzielone przez wiele wątków i są te dane zmieniane między wątkami.

Modyfikacja głównej pętli

```
void mandelbrot(int tid){  
    int myiy = 0;  
    while(myiy<iYmax){  
        mtx.lock();  
        myiy = counter++;  
        mtx.unlock();  
        int iY=myiy;  
  
        if(iY < iYmax){  
            double Cy=CyMin + iY*PixelHeight;  
            if (fabs(Cy)< PixelHeight/2) Cy=0.0;  
            int Iteration=0;
```

Suma iteracji przy pierwszej kompilacji

```
Dla 4  
Dla wątko 1 Suma iteracji:558640177  
Dla wątko 2 Suma iteracji:595506712  
Dla wątko 3 Suma iteracji:556454436  
Dla wątko 4 Suma iteracji:600050024  
5.69378  
  
Dla 8  
Dla wątko 1 Suma iteracji:288663560  
Dla wątko 2 Suma iteracji:288660991  
Dla wątko 3 Suma iteracji:288578266  
Dla wątko 4 Suma iteracji:289919436  
Dla wątko 5 Suma iteracji:288840762  
Dla wątko 6 Suma iteracji:289265384  
Dla wątko 7 Suma iteracji:287992508  
Dla wątko 8 Suma iteracji:288730442  
3.49716
```

Suma iteracji przy drugiej kompilacji (testowo)

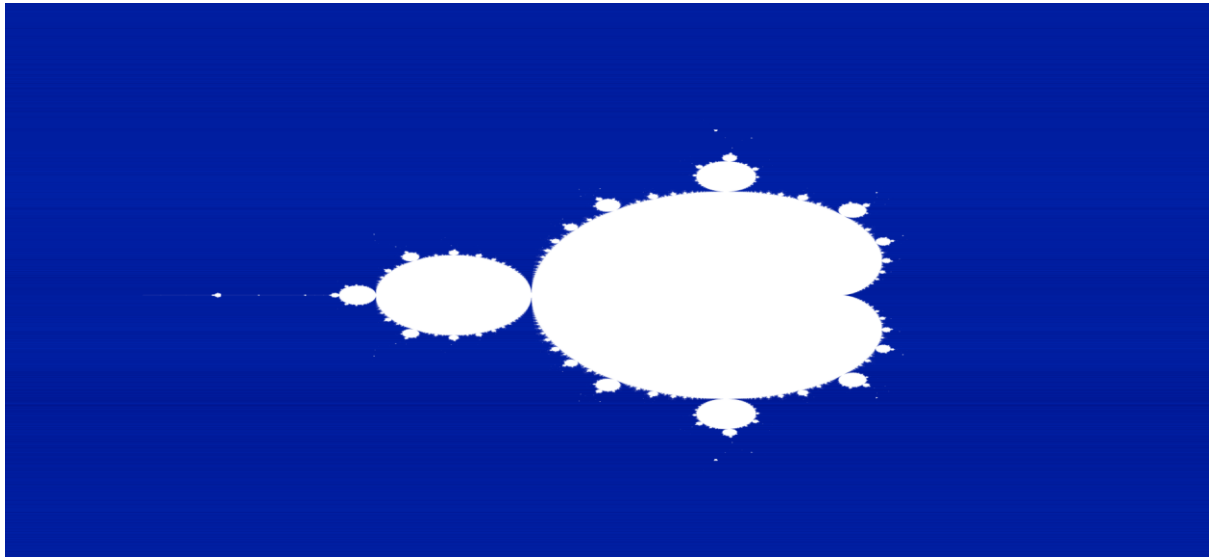
```
Dla 4
Dla wątku 1 Suma iteracji:568081234
Dla wątku 2 Suma iteracji:584653723
Dla wątku 3 Suma iteracji:586096247
Dla wątku 4 Suma iteracji:571820145
5.70472

Dla 8
Dla wątku 1 Suma iteracji:290853414
Dla wątku 2 Suma iteracji:286486889
Dla wątku 3 Suma iteracji:290864303
Dla wątku 4 Suma iteracji:289890918
Dla wątku 5 Suma iteracji:287753331
Dla wątku 6 Suma iteracji:291422587
Dla wątku 7 Suma iteracji:287090051
Dla wątku 8 Suma iteracji:286289856
3.70108
```

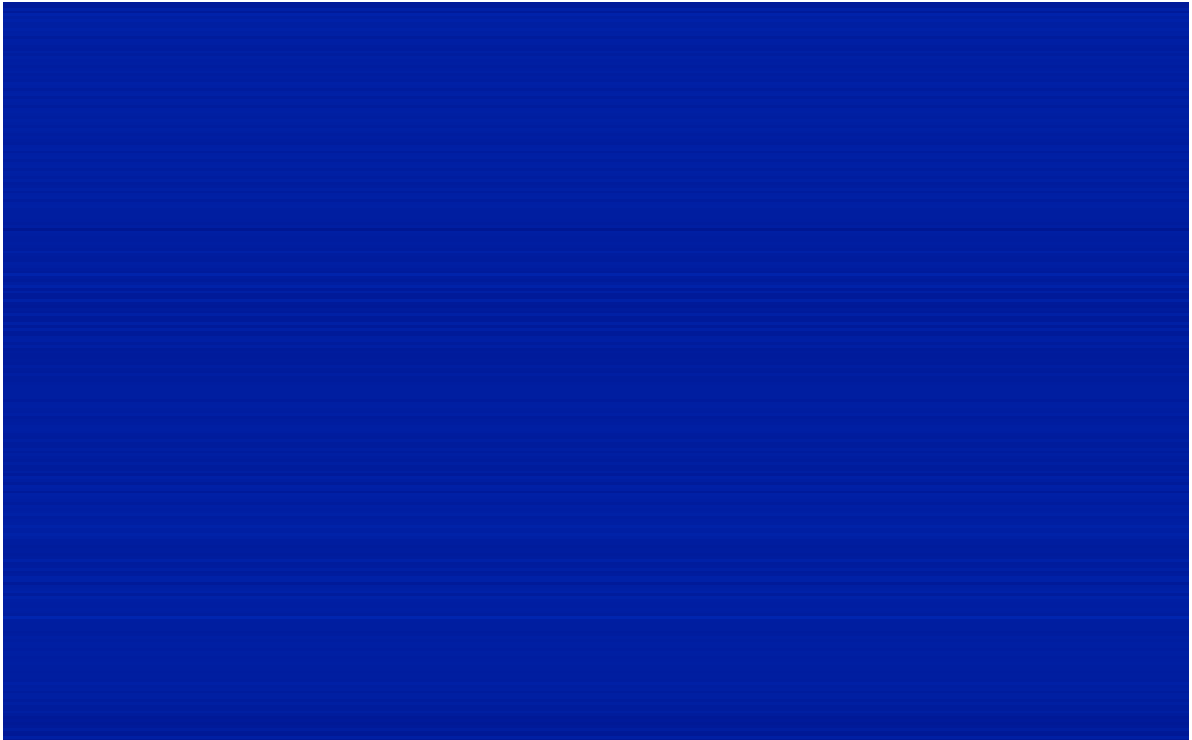
Czasy porównanie

N	1	2	4	8	16	32	64	128	256	512
METODA PODZIAŁ NA BLOKI	16.9677	8,4067	8.54225	7.30516	4.87363	3.47504	3.00439	2.87545	2.85453	2.85595
METODA ROLETY	17.4244	9,68922	5.79364	3.66359	3,3541	3.15754	3.18418	3.29687	3.06062	2,8987
DYNAMICZNE PRZYDZIELANIE	17.4628	10,3958	5,69378	3.49716	3.42416	3.23092	2.95712	2.93156	3.06007	2.90809

Obraz



Obraz w przybliżeniu



Omówienie wyników

Czasy delikatnie są lepsze niż dla metody rolety. Możemy zauważyć, że ilość wykonanych iteracji dla wątków zmienia się przy każdym skompilowaniu. Jest to spowodowane tym, że przydział wierszy nie jest deterministyczny. Za każdym uruchomieniem wątki dostają inne wiersze, bo mutex przydziela je dynamicznie w zależności od tego, który wątek akurat pierwszy zdąży je pobrać. Na przybliżonym obrazie możemy zauważyć, że kolory nie są rozłożone tak równomiernie jak przy metodzie rolety co jest typowym zachowaniem dla przydziału dynamicznego.